# CSC3022F DPRMAR021 Machine Learning Assignment 1

Mark Du Preez

May 2025

Designing a Feed-Forward Neural Network to Classify Images into Fashion Product Categories from the FashionMNIST Dataset.

## 1 Introduction

The aim of this assignment is to develop a feed-forward neural network capable of accurately classifying images from the FashionMNIST dataset into their corresponding fashion product categories. The design process involves determining an appropriate network architecture and selecting suitable hyperparameters. This report provides a detailed evaluation of the design choices made, with particular emphasis on the selected hyperparameters and the reasoning behind these decisions.

## 2 Implementation

The neural network was implemented in Python using the `PyTorch` and `torchvision` libraries. The dataset used was FashionMNIST, comprising of 60,000 training images and 10,000 test images. The training set was further partitioned into 48,000 training and 12,000 validation samples. The validation set was used to monitor model performance during training by computing the validation accuracy at the end of each epoch which is also used in the early stopping policy discussed below.

To enhance generalisation and reduce overfitting, early stopping was employed. Training was halted if validation accuracy did not improve for 20 consecutive epochs - an experimentally determined threshold optimised for the selected batch size (discussed later).

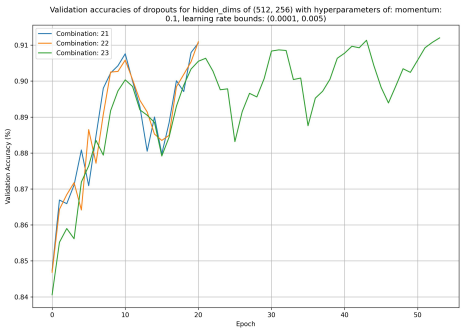
Hyperparameter optimisation was conducted via an extensive grid search using the `optimize.py` script. This process involved exploring combinations of architectural and training parameters, including cyclical learning rate bounds, hidden layer configurations, dropout probabilities, batch sizes, and batch normalisation momentum. After each search, the most consistently effective configurations were refined iteratively. This approach led to a final model achieving

a test accuracy of **90.81%**. The final hyperparameters are detailed in later sections.

The core implementation, `classifier.py`, reflects the optimal configuration identified through this process.

# 3 Neural Network Topology

## 3.1 Hidden layers




(a) Validation Accuracy vs Epochs

Figure 1: Validation accuracy curves for hidden dimensions of 512 and 256 for 3 different combinations of hyperparameters

The final feed-forward neural network architecture consists of a fully connected input layer of size **784**, two hidden layers of sizes **512** and **256**, and an output layer of size **10**. This structure was guided in part by recommendations suggesting that 1–5 layers are typically sufficient for image classification tasks of moderate and high complexity (Sachdev 2025).

To determine an optimal architecture, a series of automated grid searches were performed using the `optimize.py` script. These searches varied architectural and training hyperparameters, including the number and size of hidden layers, cyclical learning rate bounds, dropout probabilities, batch sizes, and momentum values.

The empirical results indicated that models with three or more hidden layers exhibited noticeable overfitting, achieving high validation accuracy but poorer test performance—even with regularisation techniques such as dropout. This aligns with expectations for the relatively simple FashionMNIST dataset, which does not demand deep architectures for effective classification. A two-hidden-layer architecture was therefore selected to balance complexity and generalisation.

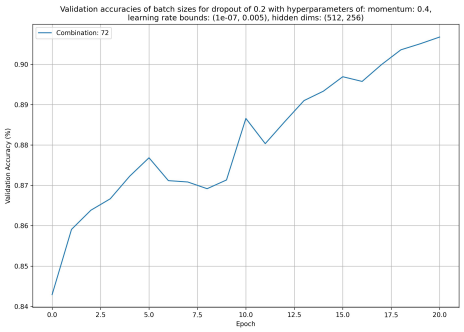
The final choice of hidden layer sizes was based on stability and performance. Among the tested configurations, 1024 – 512 – 256, 512 – 256, and 256 – 128.

The 512 – 256 setup, consistently yielded the most stable validation accuracy and best generalisation, as shown in Figure 1a.

## 3.2 Activation Function

The selection of activation functions was informed by the recommendations of Pragati Baheti, who provides a detailed overview of various activation functions and their appropriate use cases (Baheti 2021). For the output layer, a softmax activation function was used, which is implicitly incorporated within the chosen loss function implementation (cross-entropy loss discussed later), as is standard for multiclass classification tasks. For the hidden layers, the Rectified Linear Unit (ReLU) was selected due to its computational efficiency and effectiveness in feed-forward neural networks (Baheti 2021). Potential drawbacks of ReLU, such as the Dying ReLU problem and issues related to vanishing or exploding gradients, were also considered. To address these concerns, batch normalisation was subsequently applied to stabilise the training process and improve gradient flow.

## 3.3 Batch Normalisation




(a) Validation Accuracy vs Epochs

Figure 2: Validation accuracy curves for a momentum of 0.4, dropout probability of 0.2, and learning rate bounds of $5 \times 10^{-8}$ to $5 \times 10^{-3}$.

During training, particularly with smaller learning rates, the validation accuracy exhibited unstable convergence. To mitigate this issue, 1D batch normalisation layers were introduced after the input and each hidden layer. This adjustment aimed to stabilise the training process by smoothing the gradient surface and promoting more consistent convergence of the validation accuracy and training loss especially for small learning rates.

Subsequent experimentation with different momentum values for batch normalisation revealed that a momentum of **0.4** yielded the most stable results for

the selected hyperparameter configuration — specifically, a dropout probability of 0.2 and learning rate bounds of $5 \times 10^{-8}$ to $5 \times 10^{-3}$. The convergence behaviour under these settings is illustrated in Figure 2.

### 3.4 Dropout Layers

To mitigate overfitting and improve generalisation, dropout was applied to the input and hidden layers. Grid search experiments explored dropout probabilities of 0.2, 0.3, 0.4, and 0.5. Results showed that lower dropout rates were generally more effective at smaller learning rates and higher batch normalisation momentums, yielding more stable convergence. Additionally, higher dropout values tended to increase convergence time. A dropout probability of **0.2** was ultimately selected as it provided the most stable training convergence of training loss and validation accuracy under the final set of hyperparameters (see Figure 2).

## 4 Loss Function

Loss functions quantify how well a neural network performs a given task, such as classification tasks (Opperman 2022). Based on the widely recommended approach of using cross-entropy loss for neural network classification problems involving probabilistic predictions, cross-entropy was selected as my loss function for my final model.

## 5 Optimizer

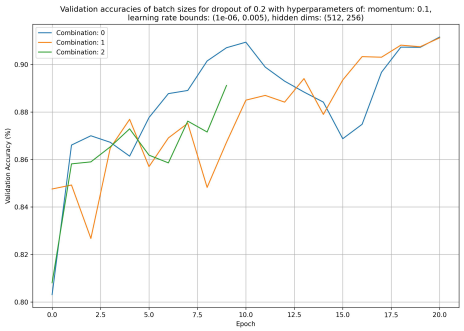
The Adam optimizer was selected due to its adaptive learning rate capabilities, bias correction, low memory requirements, and its ability to achieve faster and more stable convergence (Vishwakarma 2025). These features make it well-suited for training deep neural networks across a range of tasks.

### 5.1 Cyclical Learning Rates

To further enhance training efficiency, a Cyclical Learning Rate (CLR) policy was implemented in conjunction with the Adam optimiser. As demonstrated by Smith (Smith 2017), CLR policies can improve classification performance without requiring extensive manual tuning of learning rates. During hyperparameter optimisation, various combinations of base and maximum learning rates were tested. A base learning rate of $1 \times 10^{-7}$ and a maximum learning rate of $5 \times 10^{-3}$ were selected, as this configuration yielded a stable and reasonably fast convergence of both training loss and validation accuracy.

# 6 Other Hyperparameters

## 6.1 Batch Size




(a) Validation Accuracy vs. Epochs

Figure 3: Validation accuracy curves for varying batch sizes using an example hyperparameter configuration from the grid search.

Batch size was an important hyperparameter explored during the grid search. After several trials, a batch size of **256** was selected in conjunction with an early stopping threshold of 20 epochs. This choice was made relatively early in the experimentation process, based on consistent observations that a batch size of 256 yielded more stable validation accuracy convergence across a variety of hyperparameter configurations.

This trend is illustrated in Figure 3a, which shows an early grid search result: combination 0 corresponds to a batch size of 128, combination 1 to a batch size of 256, and combination 2 to a batch size of 1024. The batch size of 256 consistently demonstrated smoother and more stable convergence patterns compared to the other values tested.

The selected batch sizes were chosen based on powers of two, following the guidance of Nielsen's 'Neural networks and deep learning', which recommends varying batch size in this manner to identify an optimal value that ensures stable convergence of validation accuracies (Nielsen 2015). This strategy ultimately led to the selection of 256 as the most effective batch size for this task.

# 7 References

# References

Baheti, P. (2021). *Activation Functions in Neural Networks [12 Types & Use Cases]*. [Accessed 26 April 2025].

Nielsen, M. A. (2015). *Neural Networks and Deep Learning*. San Francisco, CA, USA: Determination Press.

Opperman, A. (2022). *How Loss Functions Work in Neural Networks and Deep Learning*. https://builtin.com/machine-learning/loss-functions. [Accessed 27 April 2025].

Sachdev, H. S. (2025). *Choosing Hidden Layers and Neurons in an MLP Model: A Practical Guide*. https://medium.com/@hss245/choosing-hidden-layers-and-neurons-in-an-mlp-model-a-practical-guide-2777ecf3ab4b. [Accessed 27 April 2025].

Smith, L. N. (Mar. 2017). "Cyclical Learning Rates for Training Neural Networks". In: *2017 IEEE Winter Conference on Applications of Computer Vision (WACV)*. Santa Rosa, CA, USA, pp. 464–472.

Vishwakarma, N. (2025). *What is Adam Optimizer?* https://www.analyticsvidhya.com/blog/2023/09/what-is-adam-optimizer/. [Accessed 26 April 2025].